

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ  
РОССИЙСКОЙ ФЕДЕРАЦИИ  
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ БЮДЖЕТНОЕ  
ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ  
«ВЯТСКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ»  
Институт математики и информационных систем  
Факультет автоматики и вычислительной техники  
Кафедра электронных вычислительных машин

Отчёт по лабораторной работе №2.2  
по дисциплине  
«Управление данными»

Выполнил студент гр. ИВТб-2302-06-00

\_\_\_\_\_/Терешенко Н.А./

Проверил преподаватель

\_\_\_\_\_/Клюкин В.Л./

Киров

2026

## Цель работы

Освоить основы DML-запросов в PostgreSQL: научиться использовать агрегатные функции с предложением `GROUP BY/HAVING`, операторы и встроенные функции, работу с датами, подзапросы и предложения `WITH` и `RETURNING`. Также рассмотреть планы выполнения запросов (методы доступа и способы соединения наборов строк), выполнить `FORMAT JSON` для выборки из нескольких связанных таблиц и, получив три способа соединения для одной и той же выборки, определить наиболее оптимальный.

## Ход работы

### Задание 1. Агрегатные функции

#### Исходный код

```
SELECT COUNT(1) FROM users;
```

```
SELECT SUM(sold_products), MIN(sold_products), MAX(sold_products), AVG(sold_products)
FROM sales_total;
```

```
SELECT
    u.user_id,
    u.first_name,
    u.last_name,
    STRING_AGG(t.tier_name, ', ') AS tiers,
    AVG(v.visits_cnt) AS average_visits
FROM users u, tiers t, visits v
WHERE v.user_id = u.user_id
    AND v.tier_id = t.tier_id
GROUP BY u.user_id, u.first_name, u.last_name
ORDER BY average_visits ASC;
```

```
SELECT
    u.user_id,
    u.first_name,
```

```

    u.last_name,
    STRING_AGG(t.tier_name, ', ') AS tiers,
    AVG(v.visits_cnt) AS average_visits
FROM users u, tiers t, visits v
WHERE v.user_id = u.user_id
    AND v.tier_id = t.tier_id
GROUP BY u.user_id, u.first_name, u.last_name
HAVING AVG(v.visits_cnt) > 4
ORDER BY average_visits ASC;

```

## Результат

|             |               |              |  |                   |  |                    |  |         |  |         |  |
|-------------|---------------|--------------|--|-------------------|--|--------------------|--|---------|--|---------|--|
| 123 count   |               | 123 sum      |  |                   |  | 123 min            |  | 123 max |  | 123 avg |  |
| 5           |               | 200          |  | 5                 |  | 120                |  | 40      |  |         |  |
| 123 user_id | AZ first_name | AZ last_name |  | AZ tiers          |  | 123 average_visits |  |         |  |         |  |
| 1           | 5             | Дмитрий      |  | Кузнецов          |  | Разовый визит      |  | 1       |  |         |  |
| 2           | 4             | Елена        |  | Смирнова          |  | Базовый + Басс     |  | 3       |  |         |  |
| 3           | 1             | Иван         |  | Иванов            |  | Базовый 1 месяц    |  | 8       |  |         |  |
| 4           | 3             | Мария        |  | Сидорова          |  | Стандарт 3 мес     |  | 12      |  |         |  |
| 5           | 2             | Алексей      |  | Петров            |  | Премиум 12 мес     |  | 42      |  |         |  |
| 123 user_id | AZ first_name | AZ last_name |  | AZ tiers          |  | 123 average_visits |  |         |  |         |  |
| 1           | Иван          | Иванов       |  | Базовый 1 месяц   |  | 8                  |  |         |  |         |  |
| 3           | Мария         | Сидорова     |  | Стандарт 3 месяца |  | 12                 |  |         |  |         |  |
| 2           | Алексей       | Петров       |  | Премиум 12 месяа  |  | 42                 |  |         |  |         |  |

## Задание 2. Операторы и встроенные функции, работа с датами

Ниже приведён сеанс psql, в котором последовательно выполнены команды с операторами и встроенными функциями, работа с датами, и показаны их результаты.

```
postgres=# SELECT NOT 1 < 2 AND 3 < 4 OR 5 = 5;
```

-----

```

t
(1 row)

```

```
postgres=# SELECT 1 <> 1 OR 5 != 5;
```

-----

```

f
(1 row)

```

```
postgres=# SELECT 1 IS NULL OR 5 IS NOT NULL;
```

```
-----  
t  
(1 row)
```

```
postgres=# SELECT 3 BETWEEN 3 AND 5;
```

```
-----  
t  
(1 row)
```

```
postgres=# SELECT 1 NOT BETWEEN 1 AND 5;
```

```
-----  
f  
(1 row)
```

```
postgres=# SELECT 'Hello, ' || 'Postgres!';
```

```
-----  
Hello, Postgres!  
(1 row)
```

```
postgres=# SELECT CONCAT('Hello, ', 'Postgres!');
```

```
concat  
-----  
Hello, Postgres!  
(1 row)
```

```
postgres=# SELECT 'abc' || NULL;
```

```
-----  
  
(1 row)
```

```
postgres=# SELECT CONCAT('abc', NULL, 'de');
```

```
concat  
-----  
abcde
```

(1 row)

```
postgres=# SELECT 'first' || CHR(13) || 'second';
```

-----

first\rsecond

(1 row)

```
postgres=# SELECT 'Don''t worry';
```

-----

Dont worry

(1 row)

```
postgres=# SELECT LOWER('AbC'), UPPER('AbC');
```

lower | upper

-----+-----

abc | ABC

(1 row)

```
postgres=# SELECT TRIM('  abc ');
```

btrim

-----

abc

(1 row)

```
postgres=# SELECT CASE WHEN 1 = 1 THEN 'AAA' ELSE '000' END;
```

case

-----

AAA

(1 row)

```
postgres=# SELECT COALESCE(NULL, NULL, 42, NULL, 33);
```

coalesce

-----

42

```
(1 row)
```

```
postgres=# SELECT 'abcdef' LIKE '%c%';
```

```
-----
```

```
      t
```

```
(1 row)
```

```
postgres=# SELECT 42::text;
```

```
      text
```

```
-----
```

```
      42
```

```
(1 row)
```

```
postgres=# SELECT NOW();
```

```
      now
```

```
-----
```

```
2026-09-13 15:21:42.151336+00
```

```
(1 row)
```

```
postgres=# SELECT '12-05-2026'::date;
```

```
      date
```

```
-----
```

```
2026-12-05
```

```
(1 row)
```

```
postgres=# SELECT TO_DATE('12-05-2026', 'DD-MM-YYYY');
```

```
      to_date
```

```
-----
```

```
2026-05-12
```

```
(1 row)
```

```
postgres=# SELECT '12-05-2026 12:08'::timestamp;
```

```
      timestamp
```

```
-----
```

```
2026-12-05 12:08:00
(1 row)
```

```
postgres=# SELECT TO_TIMESTAMP('12-05-2026 12:08', 'DD-MM-YYYY HH24:MI');
          to_timestamp
-----
2026-05-12 12:08:00+00
(1 row)
```

```
postgres=# SELECT TO_DATE('12-05-2026', 'DD-MM-YYYY') + 1;
-----
2026-05-13
(1 row)
```

```
postgres=# SELECT TO_DATE('12-05-2026', 'DD-MM-YYYY') + INTERVAL '1 day';
-----
2026-05-13 00:00:00
(1 row)
```

```
postgres=# SELECT TO_DATE('12-05-2026', 'DD-MM-YYYY') - INTERVAL '5 year';
-----
2021-05-12 00:00:00
(1 row)
```

```
postgres=# SELECT TO_TIMESTAMP('12-05-2026 12:08', 'DD-MM-YYYY HH24:MI') + INTERVAL '1 year';
-----
2027-07-12 12:08:11+00
(1 row)
```

```
postgres=# SELECT TO_DATE('12-05-2026', 'DD-MM-YYYY') - TO_DATE('10-05-2026', 'DD-MM-YYYY');
-----
2
(1 row)
```

### Задание 3. Подзапросы и предложение WITH

#### Исходный код

```
SELECT * FROM (SELECT * FROM users) s;
```

```
DELETE FROM users
```

```
WHERE user_id IN (SELECT user_id FROM visits WHERE visits_cnt = 3);
```

```
SELECT first_name, last_name
```

```
FROM users
```

```
WHERE user_id IN (SELECT user_id FROM visits WHERE on_hold = TRUE);
```

```
DROP TABLE IF EXISTS active_subs CASCADE;
```

```
CREATE TABLE public.active_subs (
```

```
    user_id BIGINT PRIMARY KEY,
```

```
    first_name VARCHAR(50) NOT NULL,
```

```
    last_name VARCHAR(50) NOT NULL,
```

```
    visits_cnt INT NOT NULL CHECK (visits_cnt >= 0),
```

```
    subtn_active BOOLEAN DEFAULT TRUE NOT NULL CHECK (subtn_active = TRUE),
```

```
    CONSTRAINT fk_active_subs_user FOREIGN KEY (user_id)
```

```
        REFERENCES public.users(user_id) ON DELETE CASCADE
```

```
);
```

```
WITH active_users AS (
```

```
    SELECT DISTINCT
```

```
        v.user_id,
```

```
        v.visits_cnt,
```

```
        v.subtn_active
```

```
    FROM public.visits v
```

```
    WHERE v.subtn_active = TRUE
```

```
)
```

```
INSERT INTO public.active_subs (user_id, first_name, last_name, visits_cnt, subtn_act
```

```
SELECT
```

```
    u.user_id,
```

```

    u.first_name,
    u.last_name,
    au.visits_cnt,
    au.subtn_active
FROM public.users u
JOIN active_users au ON u.user_id = au.user_id;

SELECT * FROM active_subs;

```

## Результат

|   | 123 user_id | AZ first_name | AZ last_name | AZ phone_number | AZ id_card_num |
|---|-------------|---------------|--------------|-----------------|----------------|
| 1 | 1           | Иван          | Иванов       | +79991112233    | 123456         |
| 2 | 2           | Алексей       | Петров       | +79992223344    | 0909           |
| 3 | 3           | Мария         | Сидорова     | +79993334455    | 345678         |
| 4 | 4           | Елена         | Смирнова     | +79994445566    | 456789         |
| 5 | 5           | Дмитрий       | Кузнецов     | +79995556677    | 567890         |

  

| user_id | first_name | last_name | phone_number | id_card_num |
|---------|------------|-----------|--------------|-------------|
| 1       | Иван       | Иванов    | +79991112233 | 123456      |
| 2       | Алексей    | Петров    | +79992223344 | 0909        |
| 3       | Мария      | Сидорова  | +79993334455 | 345678      |
| 5       | Дмитрий    | Кузнецов  | +79995556677 | 567890      |

  

| AZ first_name | AZ last_name |
|---------------|--------------|
| Мария         | Сидорова     |

  

|   | 123 user_id | AZ first_name | AZ last_name | 123 visits_cnt | subtn_active |
|---|-------------|---------------|--------------|----------------|--------------|
| 1 | 4           | Елена         | Смирнова     | 3              | [v]          |
| 2 | 1           | Иван          | Иванов       | 8              | [v]          |
| 3 | 2           | Алексей       | Петров       | 42             | [v]          |

## Задание 4. Предложение RETURNING

С помощью предложения RETURNING команда UPDATE возвращает изменённые строки.

### Исходный код

```

UPDATE product
SET sale_price = sale_price - 50
WHERE sale_price > 150
RETURNING *;

```

## Результат

|   | 123 product_id ▼ | 123 sale_price ▼ |
|---|------------------|------------------|
| 1 | 2                | 150              |
| 2 | 3                | 300              |
| 3 | 4                | 1,150            |
| 4 | 5                | 350              |

## Задание 5. Планы выполнения запросов

Для получения плана выполнения запроса используется команда `EXPLAIN`. Допускаются параметры `ANALYZE`, `VERBOSE`, `COSTS`, `BUFFERS`, `TIMING` и `FORMAT {TEXT | XML | JSON | YAML}`. `ANALYZE` выполняет команду и выводит фактическое время выполнения, `VERBOSE` — дополнительную информацию о плане (выводимые столбцы и полные имена таблиц), `BUFFERS` — сведения об использовании буферного кэша. По умолчанию планы выводятся в текстовом формате.

### Исходный код

```
EXPLAIN (ANALYZE) SELECT * FROM users;
EXPLAIN (VERBOSE) SELECT * FROM users;
EXPLAIN (COSTS) SELECT * FROM users;
EXPLAIN (ANALYZE, BUFFERS) SELECT * FROM users;
EXPLAIN (ANALYZE, TIMING) SELECT * FROM users;
```

## Результат

### QUERY PLAN

```
-----
Seq Scan on users  (cost=0.00..11.80 rows=180 width=420) (actual time=0.005.
.0.006 rows=5.00 loops=1)
  Buffers: shared hit=1
Planning:
  Buffers: shared hit=82
Planning Time: 0.265 ms
Execution Time: 0.016 ms
(6 rows)
```

#### QUERY PLAN

```
-----  
Seq Scan on public.users (cost=0.00..11.80 rows=180 width=420)  
  Output: user_id, first_name, last_name, phone_number, id_card_num  
(2 rows)
```

#### QUERY PLAN

```
-----  
Seq Scan on users (cost=0.00..11.80 rows=180 width=420)  
(1 row)
```

#### QUERY PLAN

```
-----  
Seq Scan on users (cost=0.00..11.80 rows=180 width=420) (actual time=0.004.  
.0.004 rows=5.00 loops=1)  
  Buffers: shared hit=1  
Planning Time: 0.006 ms  
Execution Time: 0.006 ms  
(4 rows)
```

#### QUERY PLAN

```
-----  
Seq Scan on users (cost=0.00..11.80 rows=180 width=420) (actual time=0.006.  
.0.006 rows=5.00 loops=1)  
  Buffers: shared hit=1  
Planning Time: 0.035 ms  
Execution Time: 0.013 ms  
(4 rows)
```

Далее рассмотрены форматы вывода XML, JSON и YAML для выборки из двух связанных таблиц (users и visits).

#### Формат XML

#### Исходный код

```

EXPLAIN (FORMAT XML)
SELECT * FROM users u
JOIN visits v ON u.user_id = v.user_id;

```

## Результат

```

<explain xmlns="http://www.postgresql.org/2009/explain">
  <Query>
    <Plan>
      <Node-Type>Hash Join</Node-Type>
      <Parallel-Aware>>false</Parallel-Aware>
      <Async-Capable>>false</Async-Capable>
      <Join-Type>Inner</Join-Type>
      <Startup-Cost>14.05</Startup-Cost>
      <Total-Cost>46.50</Total-Cost>
      <Plan-Rows>1770</Plan-Rows>
      <Plan-Width>438</Plan-Width>
      <Disabled>>false</Disabled>
      <Inner-Unique>>true</Inner-Unique>
      <Hash-Cond>(v.user_id = u.user_id)</Hash-Cond>
      <Plans>
        <Plan>
          <Node-Type>Seq Scan</Node-Type>
          <Parent-Relationship>Outer</Parent-Relationship>
          <Parallel-Aware>>false</Parallel-Aware>
          <Async-Capable>>false</Async-Capable>
          <Relation-Name>visits</Relation-Name>
          <Alias>v</Alias>
          <Startup-Cost>0.00</Startup-Cost>
          <Total-Cost>27.70</Total-Cost>
          <Plan-Rows>1770</Plan-Rows>
          <Plan-Width>18</Plan-Width>
          <Disabled>>false</Disabled>
        </Plan>
      <Plan>

```

```

    <Node-Type>Hash</Node-Type>
    <Parent-Relationship>Inner</Parent-Relationship>
    <Parallel-Aware>false</Parallel-Aware>
    <Async-Capable>false</Async-Capable>
    <Startup-Cost>11.80</Startup-Cost>
    <Total-Cost>11.80</Total-Cost>
    <Plan-Rows>180</Plan-Rows>
    <Plan-Width>420</Plan-Width>
    <Disabled>false</Disabled>
    <Plans>
      <Plan>
        <Node-Type>Seq Scan</Node-Type>
        <Parent-Relationship>Outer</Parent-Relationship>
        <Parallel-Aware>false</Parallel-Aware>
        <Async-Capable>false</Async-Capable>
        <Relation-Name>users</Relation-Name>
        <Alias>u</Alias>
        <Startup-Cost>0.00</Startup-Cost>
        <Total-Cost>11.80</Total-Cost>
        <Plan-Rows>180</Plan-Rows>
        <Plan-Width>420</Plan-Width>
        <Disabled>false</Disabled>
      </Plan>
    </Plans>
  </Plan>
</Plans>
</Plan>
</Query>
</explain>
(1 row)

```

**Формат JSON**

**Исходный код**

**EXPLAIN (FORMAT JSON)**

```
SELECT * FROM users u
JOIN visits v ON u.user_id = v.user_id;
```

## Результат

```
[
  {
    "Plan": {
      "Node Type": "Hash Join",
      "Parallel Aware": false,
      "Async Capable": false,
      "Join Type": "Inner",
      "Startup Cost": 14.05,
      "Total Cost": 46.50,
      "Plan Rows": 1770,
      "Plan Width": 438,
      "Disabled": false,
      "Inner Unique": true,
      "Hash Cond": "(v.user_id = u.user_id)",
      "Plans": [
        {
          "Node Type": "Seq Scan",
          "Parent Relationship": "Outer",
          "Parallel Aware": false,
          "Async Capable": false,
          "Relation Name": "visits",
          "Alias": "v",
          "Startup Cost": 0.00,
          "Total Cost": 27.70,
          "Plan Rows": 1770,
          "Plan Width": 18,
          "Disabled": false
        },
        {
          "Node Type": "Hash",
```

```

    "Parent Relationship": "Inner",
    "Parallel Aware": false,
    "Async Capable": false,
    "Startup Cost": 11.80,
    "Total Cost": 11.80,
    "Plan Rows": 180,
    "Plan Width": 420,
    "Disabled": false,
    "Plans": [
      {
        "Node Type": "Seq Scan",
        "Parent Relationship": "Outer",
        "Parallel Aware": false,
        "Async Capable": false,
        "Relation Name": "users",
        "Alias": "u",
        "Startup Cost": 0.00,
        "Total Cost": 11.80,
        "Plan Rows": 180,
        "Plan Width": 420,
        "Disabled": false
      }
    ]
  }
]
}
]
}
}
]

```

## Формат YAML

### Исходный код

```

EXPLAIN (FORMAT YAML)
SELECT * FROM users u
JOIN visits v ON u.user_id = v.user_id;

```

## Результат

### Plan:

Node Type: "Hash Join"  
Parallel Aware: false  
Async Capable: false  
Join Type: "Inner"  
Startup Cost: 14.05  
Total Cost: 46.50  
Plan Rows: 1770  
Plan Width: 438  
Disabled: false  
Inner Unique: true  
Hash Cond: "(v.user\_id = u.user\_id)"

### Plans:

- Node Type: "Seq Scan"
  - Parent Relationship: "Outer"
  - Parallel Aware: false
  - Async Capable: false
  - Relation Name: "visits"
  - Alias: "v"
  - Startup Cost: 0.00
  - Total Cost: 27.70
  - Plan Rows: 1770
  - Plan Width: 18
  - Disabled: false
- Node Type: "Hash"
  - Parent Relationship: "Inner"
  - Parallel Aware: false
  - Async Capable: false
  - Startup Cost: 11.80
  - Total Cost: 11.80
  - Plan Rows: 180
  - Plan Width: 420
  - Disabled: false

Plans:

- Node Type: "Seq Scan"
  - Parent Relationship: "Outer"
  - Parallel Aware: false
  - Async Capable: false
  - Relation Name: "users"
  - Alias: "u"
  - Startup Cost: 0.00
  - Total Cost: 11.80
  - Plan Rows: 180
  - Plan Width: 420
  - Disabled: false

## Задание 6. Методы доступа

Методы доступа характеризуют способ просмотра таблиц. Последовательный просмотр (Sequential Scan) применяется, когда требуется выбрать все строки таблицы или значительную их часть. Index Scan — просмотр на основе индекса, целесообразен при высокой селективности выборки. Index Only Scan не обращается к строкам таблицы, если все запрашиваемые столбцы присутствуют в индексе. Bitmap Scan формирует битовую карту страниц и обращается к каждой странице только один раз.

### Исходный код

```
EXPLAIN (FORMAT JSON) SELECT * FROM users;
```

```
EXPLAIN (FORMAT JSON) SELECT * FROM users WHERE user_id = 2;
```

```
EXPLAIN (FORMAT JSON) SELECT user_id FROM users WHERE user_id = 2;
```

```
EXPLAIN (FORMAT JSON) SELECT * FROM product WHERE sale_price > 150;
```

### Результат

QUERY PLAN

-----

```
[
  {
    "Plan": {
      "Node Type": "Seq Scan",
      "Parallel Aware": false,
      "Async Capable": false,
      "Relation Name": "users",
      "Alias": "users",
      "Startup Cost": 0.00,
      "Total Cost": 11.80,
      "Plan Rows": 180,
      "Plan Width": 420,
      "Disabled": false
    }
  }
]
(1 row)
```

#### QUERY PLAN

```
-----
[
  {
    "Plan": {
      "Node Type": "Index Scan",
      "Parallel Aware": false,
      "Async Capable": false,
      "Scan Direction": "Forward",
      "Index Name": "users_pkey",
      "Relation Name": "users",
      "Alias": "users",
      "Startup Cost": 0.14,
      "Total Cost": 8.16,
      "Plan Rows": 1,
      "Plan Width": 420,
```

```

        "Disabled": false,
        "Index Cond": "(user_id = 2)"
    }
}
]
(1 row)

```

#### QUERY PLAN

```

-----
[
  {
    "Plan": {
      "Node Type": "Index Only Scan",
      "Parallel Aware": false,
      "Async Capable": false,
      "Scan Direction": "Forward",
      "Index Name": "users_pkey",
      "Relation Name": "users",
      "Alias": "users",
      "Startup Cost": 0.14,
      "Total Cost": 8.16,
      "Plan Rows": 1,
      "Plan Width": 8,
      "Disabled": false,
      "Index Cond": "(user_id = 2)"
    }
  }
]
(1 row)

```

#### QUERY PLAN

```

-----
[
  {

```

```

    "Plan": {
      "Node Type": "Seq Scan",
      "Parallel Aware": false,
      "Async Capable": false,
      "Relation Name": "product",
      "Alias": "product",
      "Startup Cost": 0.00,
      "Total Cost": 29.62,
      "Plan Rows": 523,
      "Plan Width": 24,
      "Disabled": false,
      "Filter": "(sale_price > '150'::numeric)"
    }
  }
]
(1 row)

```

## Задание 7. Способы соединения наборов строк

Существует три способа соединения наборов строк. **Nested Loop** (вложенный цикл) перебирает строки внешнего набора и для каждой ищет соответствующие строки во внутреннем; эффективен для небольших выборок. **Hash Join** (соединение хешированием) помещает строки одного набора в хеш-таблицу и работает только для эквисоединений; эффективен для больших выборок. **Merge Join** (соединение методом слияния) требует предварительной сортировки обоих наборов по столбцам соединения и также работает для эквисоединений. Для одной и той же выборки из двух связанных таблиц последовательно принудительно включаются и отключаются используемые методы, чтобы получить план с каждым из них.

### Исходный код

```

SET enable_mergejoin = OFF;
SET enable_hashjoin = OFF;
EXPLAIN (FORMAT JSON)
SELECT * FROM users u

```

```
JOIN visits v ON u.user_id = v.user_id;
```

```
SET enable_hashjoin = ON;
```

```
EXPLAIN (FORMAT JSON)
```

```
SELECT * FROM users u
```

```
JOIN visits v ON u.user_id = v.user_id;
```

```
SET enable_mergejoin = ON;
```

```
SET enable_hashjoin = OFF;
```

```
EXPLAIN (FORMAT JSON)
```

```
SELECT * FROM users u
```

```
JOIN visits v ON u.user_id = v.user_id;
```

```
RESET enable_mergejoin;
```

```
RESET enable_hashjoin;
```

## Результат

### Nested Loop

#### QUERY PLAN

```
-----  
[                                                                 +  
  {                                                                 +  
    "Plan": {                                                                 +  
      "Node Type": "Nested Loop",                                                                 +  
      "Parallel Aware": false,                                                                 +  
      "Async Capable": false,                                                                 +  
      "Join Type": "Inner",                                                                 +  
      "Startup Cost": 0.14,                                                                 +  
      "Total Cost": 362.93,                                                                 +  
      "Plan Rows": 1770,                                                                 +  
      "Plan Width": 438,                                                                 +  
      "Disabled": false,                                                                 +  
      "Inner Unique": true,                                                                 +  
      "Plans": [                                                                 +
```

```

{
    "Node Type": "Seq Scan",
    "Parent Relationship": "Outer",
    "Parallel Aware": false,
    "Async Capable": false,
    "Relation Name": "visits",
    "Alias": "v",
    "Startup Cost": 0.00,
    "Total Cost": 27.70,
    "Plan Rows": 1770,
    "Plan Width": 18,
    "Disabled": false
},
{
    "Node Type": "Index Scan",
    "Parent Relationship": "Inner",
    "Parallel Aware": false,
    "Async Capable": false,
    "Scan Direction": "Forward",
    "Index Name": "users_pkey",
    "Relation Name": "users",
    "Alias": "u",
    "Startup Cost": 0.14,
    "Total Cost": 0.19,
    "Plan Rows": 1,
    "Plan Width": 420,
    "Disabled": false,
    "Index Cond": "(user_id = v.user_id)"
}
]
}
}
]
(1 row)

```

## Hash Join

### QUERY PLAN

|                                         |   |
|-----------------------------------------|---|
| -----                                   |   |
| [                                       | + |
| {                                       | + |
| "Plan": {                               | + |
| "Node Type": "Hash Join",               | + |
| "Parallel Aware": false,                | + |
| "Async Capable": false,                 | + |
| "Join Type": "Inner",                   | + |
| "Startup Cost": 14.05,                  | + |
| "Total Cost": 46.50,                    | + |
| "Plan Rows": 1770,                      | + |
| "Plan Width": 438,                      | + |
| "Disabled": false,                      | + |
| "Inner Unique": true,                   | + |
| "Hash Cond": "(v.user_id = u.user_id)", | + |
| "Plans": [                              | + |
| {                                       | + |
| "Node Type": "Seq Scan",                | + |
| "Parent Relationship": "Outer",         | + |
| "Parallel Aware": false,                | + |
| "Async Capable": false,                 | + |
| "Relation Name": "visits",              | + |
| "Alias": "v",                           | + |
| "Startup Cost": 0.00,                   | + |
| "Total Cost": 27.70,                    | + |
| "Plan Rows": 1770,                      | + |
| "Plan Width": 18,                       | + |
| "Disabled": false                       | + |
| },                                      | + |
| {                                       | + |
| "Node Type": "Hash",                    | + |
| "Parent Relationship": "Inner",         | + |

```

    "Parallel Aware": false,          +
    "Async Capable": false,          +
    "Startup Cost": 11.80,            +
    "Total Cost": 11.80,              +
    "Plan Rows": 180,                 +
    "Plan Width": 420,                +
    "Disabled": false,                +
    "Plans": [                        +
      {                               +
        "Node Type": "Seq Scan",      +
        "Parent Relationship": "Outer", +
        "Parallel Aware": false,      +
        "Async Capable": false,      +
        "Relation Name": "users",     +
        "Alias": "u",                 +
        "Startup Cost": 0.00,          +
        "Total Cost": 11.80,          +
        "Plan Rows": 180,             +
        "Plan Width": 420,            +
        "Disabled": false             +
      }                               +
    ]                                  +
  }                                    +
]                                     +
}                                     +
]                                     +
}                                     +
]                                     +
(1 row)

```

## Merge Join

### QUERY PLAN

```

-----
[                                     +
  {                                   +
    "Plan": {                         +

```

```

"Node Type": "Merge Join",      +
"Parallel Aware": false,      +
"Async Capable": false,      +
"Join Type": "Inner",        +
"Startup Cost": 141.73,      +
"Total Cost": 167.41,        +
"Plan Rows": 1770,          +
"Plan Width": 438,          +
"Disabled": false,          +
"Inner Unique": false,      +
"Merge Cond": "(u.user_id = v.user_id)", +
"Plans": [                  +
  {                          +
    "Node Type": "Sort",      +
    "Parent Relationship": "Outer", +
    "Parallel Aware": false,  +
    "Async Capable": false,  +
    "Startup Cost": 18.54,    +
    "Total Cost": 18.99,    +
    "Plan Rows": 180,        +
    "Plan Width": 420,        +
    "Disabled": false,        +
    "Sort Key": ["u.user_id"], +
    "Plans": [              +
      {                    +
        "Node Type": "Seq Scan", +
        "Parent Relationship": "Outer", +
        "Parallel Aware": false, +
        "Async Capable": false, +
        "Relation Name": "users", +
        "Alias": "u",          +
        "Startup Cost": 0.00,   +
        "Total Cost": 11.80,   +
        "Plan Rows": 180,      +

```

```

        "Plan Width": 420,
        "Disabled": false
    }
]
},
{
    "Node Type": "Sort",
    "Parent Relationship": "Inner",
    "Parallel Aware": false,
    "Async Capable": false,
    "Startup Cost": 123.19,
    "Total Cost": 127.61,
    "Plan Rows": 1770,
    "Plan Width": 18,
    "Disabled": false,
    "Sort Key": ["v.user_id"],
    "Plans": [
        {
            "Node Type": "Seq Scan",
            "Parent Relationship": "Outer",
            "Parallel Aware": false,
            "Async Capable": false,
            "Relation Name": "visits",
            "Alias": "v",
            "Startup Cost": 0.00,
            "Total Cost": 27.70,
            "Plan Rows": 1770,
            "Plan Width": 18,
            "Disabled": false
        }
    ]
}
]
}

```

```
}
+
]
(1 row)
```

## Вывод

В ходе выполнения лабораторной работы на базе данных фитнес-центра освоены основы DML-запросов в PostgreSQL, включая агрегатные функции, подзапросы, общие табличные выражения (`WITH`), предложение `RETURNING` и работу с датами. С помощью команды `EXPLAIN` проанализированы различные форматы, методы доступа и способы соединения строк, среди которых наименьшую стоимость и оптимальность показал `Hash Join`.